

Pirâmide de Testes

Um guia prático de 5 páginas para equilibrar velocidade, confiança e custo na automação de testes de software.



Base larga = feedback barato. Topo estreito = confiança no fluxo real.

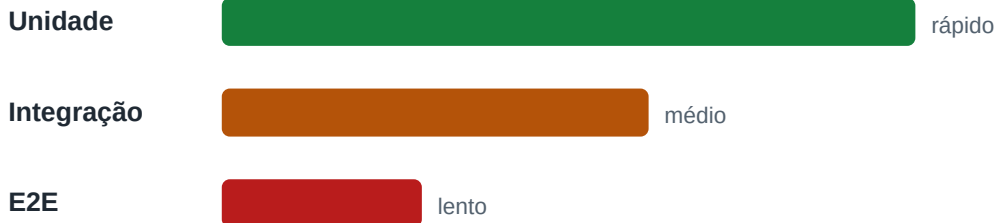
Ideia central

A pirâmide recomenda muitos testes pequenos na base, uma camada média de testes de integração e poucos testes ponta a ponta no topo.

Ela não é uma regra estética: é uma estratégia econômica. Quanto mais perto da interface real, maior o custo de escrita, execução e manutenção. Quanto mais perto da lógica isolada, mais rápido é o feedback.

1. Por que a pirâmide existe

Times costumam descobrir bugs tarde demais quando dependem quase só de testes manuais ou de testes automatizados pesados. A pirâmide organiza a suíte para responder a três perguntas: o código faz o que deveria, as partes conversam corretamente e o usuário consegue completar fluxos críticos?



O objetivo é criar ^{feedback}confiança progressiva. Testes de unidade protegem regras e decisões. Testes de integração validam banco, APIs, filas, arquivos e contratos. Testes ponta a ponta simulam jornadas reais, mas devem ser reservados ao que realmente define valor para o negócio. ^{mais cedo}

Regra prática

Se um cenário pode ser validado de forma confiável em uma camada mais baixa, prefira essa camada. Suba na pirâmide apenas quando a interação entre componentes for parte essencial do risco.

Uma boa suíte falha cedo, explica bem o problema e não transforma cada alteração simples em uma tarde de manutenção.

2. As três camadas

Testes de unidade

Verificam funções, classes, componentes ou módulos isolados. São rápidos, numerosos e ideais para cobrir regras de negócio, validações, cálculos, transformações e casos de borda.

Testes de integração

Confirmam que duas ou mais partes funcionam juntas: aplicação e banco, serviço e API externa simulada, produtor e consumidor de eventos, backend e camada de persistência. Eles expõem erros de contrato que testes de unidade não enxergam.

Testes ponta a ponta

Executam fluxos inteiros pela perspectiva do usuário, normalmente via navegador ou cliente real. São valiosos para login, compra, cadastro, pagamento, permissões e jornadas que precisam provar que o sistema completo está vivo.

Sinal de equilíbrio

A base deve rodar em segundos ou poucos minutos. A camada média pode ser um pouco mais lenta. O topo deve ser pequeno, estável e focado nos caminhos que não podem quebrar em produção.

3. Como aplicar no dia a dia

Comece por riscos, não por porcentagens rígidas. Para cada funcionalidade, pergunte: qual regra pode quebrar, qual contrato pode falhar e qual jornada do usuário precisa permanecer intacta?

Um fluxo de checkout, por exemplo, pode ter dezenas de testes de unidade para preço, desconto, frete e impostos; alguns testes de integração para estoque, pagamento e persistência; e poucos testes E2E cobrindo compra aprovada, pagamento recusado e recuperação de erro.

Checklist de decisão

1. A regra é pura? Teste de unidade. 2. Depende de banco, API, fila ou arquivo? Teste de integração. 3. Depende da experiência completa do usuário? Teste E2E.

Também vale manter dados de teste pequenos e previsíveis, evitar dependência de ordem entre testes, isolar relógio e aleatoriedade, e escrever nomes que contem o comportamento esperado.

A pirâmide funciona melhor quando faz parte do desenvolvimento, não quando aparece só no fim da sprint.

4. Armadilhas comuns e um plano simples

A primeira armadilha é a 'casquinha de sorvete': muitos testes E2E no topo e poucos testes rápidos na base. Isso deixa a suíte lenta e frágil. A segunda é testar detalhes de implementação demais, criando testes que quebram quando o código melhora sem mudar comportamento.

Outra armadilha é ignorar contratos externos. Quando uma API, fila ou esquema de banco muda, testes de unidade não bastam. A camada de integração precisa proteger essas fronteiras.

Plano em 7 dias

Dia 1: liste fluxos críticos. Dia 2: identifique regras puras. Dia 3: cubra regras com unidade. Dia 4: adicione integração nas fronteiras. Dia 5: escolha 3 a 5 E2E essenciais. Dia 6: rode tudo no CI. Dia 7: remova testes duplicados e instáveis.

Métrica saudável: a suíte deve aumentar confiança sem virar um imposto sobre cada mudança. Quando um teste falha, a mensagem precisa apontar para uma causa provável, não apenas para uma tela quebrada.

Resumo final: escreva muitos testes baratos, alguns testes de contrato e poucos testes de jornada. Essa combinação dá velocidade para mudar e segurança para entregar.